Student Course Registration System
Project Documentation

Course:: Intro to Python
Framework: Flask 3.1.3
Language: Python 3
Frontend: HTML, CSS, JavaScript
Database: SQLite (SQL)
Teammates: Ghazal Ahmadzai, Faiza Hussaini, Noorullah Ahmadi, Parwana Jafari, Sana Hashemi

Table of Contents

1. Project Overview

This project is a web-based Student Course Registration System built using Python Flask and SQLite. It supports two types of users: Admins and Students. Admins can manage courses and students, while students can browse, search, and enroll in courses. The system also tracks enrollment history for each student.

Core features:

- User registration and login with role-based access
- Admin can add, edit, and delete courses
- Students can search, enroll in, and drop courses
- Enrollment history tracking per student
- Admin dashboard with system statistics

2. Project Structure

The project is organized into one main folder with separate modules for routes, helpers, forms, and models.

The **routes/** folder contains separate files for each feature area: auth_routes.py handles login, register, and logout; course_routes.py handles course management; enrollment_routes.py handles enrollment; main_routes.py handles dashboards; and student_routes.py handles student management.

The **helpers/** folder contains db.py which handles all database connection and query helper functions.
The **forms/** folder contains forms.py with LoginForm and RegisterForm. The **models.py** file contains eusable database query functions.

```
Python- Final Project/
app.py                    # Main application file
 models.py               # Reusable database query functions
database.db              # SQLite database
 routes/                  # Route blueprints
    auth_routes.py
    course_routes.py
    enrollment_routes.py
    main_routes.py
    student_routes.py
 helpers/
    db.py                 # Database helpers
 forms/
    forms.py              # Login and register forms
 static/css/main.css
static/js/main.js
templates/               # HTML templates
```

Supporting files (Database & Testing folder):

```
Database Schema & Testing/
├── registration.py         # Database schema creation script
├── reg_insert_data.py      # Sample data insertion script
├── test_database.py        # Database unit tests
└── database.db             # SQLite database
```

3. Setup & Installation

Prerequisites

- Python 3.8 or higher
- pip (Python package manager)

Step 1 — Clone the Repository

```
git clone https://github.com/ENTJ-CYBER/course-registration-system1.git
cd course-registration-system1
```

Step 2 — Create a Virtual Environment

```
python -m venv.venv
```

```
Windows: .venv\Scripts\activate
macOS/Linux: source .venv/bin/activate
```

Activate it:

Windows: `.venv\Scripts\activate`

macOS/Linux: `source .venv/bin/activate`

Step 3 — Install Dependencies

```
pip install flask
```

Step 4 — Initialize the Database

```
cd "Python- Final Project"
python app.py
```

Step 5 — Run the Application

The app will start at: http://127.0.0.1:5000

Default Admin Login

Email: admin1@gmail.com

Password: 1111

4. Database Schema

The database uses SQLite and has four main tables:
Users: Stores all accounts (admin and student) with ID, username, email, password, and role ("admin" or "student").
Students: Contains additional details for student users, linked to the users table by user ID, including full name and department.
Courses: The courses table lists all courses with details such as course code, course name, instructor, credits, schedule, department, capacity, and description.
Enrollments: Links students to courses, storing student ID, course ID, and enrollment status (e.g., "enrolled" or "dropped"). One student can enroll in many courses, and each course can have many students.

Table: `users`

Stores all user accounts (both admins and students).

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique user ID
username	TEXT		Display name
email	TEXT	UNIQUE	Login email
password	TEXT		Hashed Password
role	TEXT		'admin' or 'student'

Table: `students`

Stores additional profile information for student users.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique student ID
user_id	INTEGER	FOREIGN KEY → users(id)	Links to the users table
full_name	TEXT		Student's full name
department	TEXT		Student's department

Table: **`Courses`**

Column	Type	Constraints	Description
id & course code	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique course ID
course_name	TEXT		Name of the course
instructor	TEXT		Instructor's name
credits	INTEGER		Credit hours
schedule	TEXT		Day/time schedule
department	TEXT		Offering department
capacity	INTEGER		Maximum enrollment
description	TEXT		Information about course

Table: **`enrollments`**

Tracks student-course relationships and enrollment status.

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique enrollment ID
student_id	INTEGER	FOREIGN KEY → students(id)	The enrolling student
course_id	INTEGER	FOREIGN KEY → courses(id)	The enrolled course
status	TEXT		`'enrolled'`, `'dropped'`, etc.

Entity Relationship Diagram (Text)

```
users (1) ──< students (1) ──< enrollments >── (1) courses
```

- One user can have one student profile
- One student can have many enrollments

- One course can have many enrollments

---

5. API Endpoints

The application has five route groups: authentication, dashboards, courses, student management, and enrollment. Authentication handles login, registration, and logout, while dashboards show system stats for admins and enrolled courses for students. Course and student routes manage data (with admin controls), and enrollment routes let students browse, search, enroll, drop courses, and view their history.

Root

Method	URL	Description
GET	`/`	Redirects to login page

Auth Blueprint (`/`)

Handles user authentication.

Method	URL	Description
GET	`/auth/login`	Display the login form
POST	`/login`	Submit login credentials; redirects to dashboard on success
GET, POST	`/auth/register`	Display and submit the student registration form
GET	`/auth/logout`	Clear session and redirect to login

Login Request Fields:

Field	Type	Required	Description
email	string	Yes	Registered email
password	string	Yes	Account password

Register Request Fields:

Field	Type	Required	Description
full_name	string	Yes	Student's full name

username	string	Yes	Display username
email	string	Yes	Email address (unique)
department	string	Yes	Student's department
password	string	Yes	Account password

Main Blueprint (/)
Handles dashboards.

Method	URL	Access	Description
GET	`/dashboard`	Admin	Admin dashboard with system statistics
GET	`/student-dashboard`	Student	Student dashboard with enrolled courses

Admin Dashboard Returns:

- Total number of students
- Total number of courses Total
- number of enrollments

  5 most recent enrollments

Courses Blueprint (`/courses`)

Handles course management. Admin-only for add/edit/delete.

Method	URL	Access	Description
GET	`/courses/`	All	List all available courses
GET, POST	`/courses/add`	Admin	Add a new course
GET	`/courses/<id>`	All	View a single course's details
GET, POST	`/courses/<id>/edit`	Admin	Edit an existing course
POST	`/courses/<id>/delete`	Admin	Delete a course

Field	Type	Description
course_name	string	Name of the course
instructor	string	Instructor name
credits	integer	Number of credit hours
schedule	string	Course schedule (e.g. Mon/Wed)
department	string	Offering department
capacity	integer	Maximum seats available

Students Blueprint (`/students`)

Handles student management. Admin-only.

Method	URL	Access	Description
GET	`/students/`	Admin	List all registered students
GET	`/students/<id>`	Admin	View a student's profile
GET, POST	`/students/<id>/edit`	Admin	Edit a student's information

Enrollment Blueprint (`/enrollment`)

Handles student course enrollment. Student-only.

Method	URL	Access	Description
GET	`/enrollment/courses`	Student	Browse available courses to enroll in
GET	`/enrollment/search`	Student	Search courses by name, instructor, or department
POST	`/enrollment/enroll/<id>`	Student	Enroll in a course
POST	`/enrollment/drop/<id>`	Student	Drop an enrolled course
GET	`/enrollment/history`	Student	View full enrollment history

Search Query Parameters:

Parameter	Type	Description
q	string	Search keyword (name, instructor, dept)

---

6. Data Models

The application stores user information in a session after login. The session holds the user's ID, username, role, full name, and email. This session data is used throughout the app to control what each user can see and do. Admins have access to course management, student management, and the admin dashboard. Students have access to course browsing, enrollment, dropping courses, and viewing their history. Neither role has access to the other's features.

Session Data

After login, the following is stored in Flask's server-side session:

Key	Description
user_id	The logged-in user's database ID
username	The user's username
role	`'admin' or 'student'`
full_name	The user's display name
email	The user's email

Role-Based Access

Feature	Admin	Student
View courses	✓	✓
Add/edit/delete courses	✓	✗
View all students	✓	✗
Edit student info	✓	✗
Enroll/drop courses	✗	✓
View enrollment history	✗	✓

Admin dashboard	✓	✗
Student dashboard	✗	✓

---

7. Testing

The project includes database tests located in the Database Schema & Testing folder. To run them, navigate to that folder and run python test_database.py.

The tests check that all four database tables — users, courses, students, and enrollments — contain data after the setup scripts have been run. Each test uses an assertion to confirm at least one row exists in the table. If everything is working, you will see four pass messages printed in the terminal, one for each table.

For manual testing, you can verify the main features by going through the app directly. Register a new student account and confirm it redirects to the login page. Log in as a student and try enrolling in and dropping a course. Log in as an admin and try adding, editing, and deleting a course. Search for a course using a keyword and confirm the results are filtered correctly.

Expected output:

```
Users test passed Courses
test passed Students test
passed Enrollments test
passed
```

Test Functions

Function	What It Tests
`test_users()`	Verifies the `users` table has at least one row
`test_courses()`	Verifies the `courses` table has at least one row
`test_students()`	Verifies the `students` table has at least one row
`test_enrollments()`	Verifies the `enrollments` table has at least one row

Each test uses success. assert to fail loudly if data is missing, and prints a pass message on

Student registration	Go to /register, fill form, submit	Redirected to login
Student login	Go to /login, enter student credentials	Redirected to student dashboard
Admin login	Enter admin credentials	Redirected to admin dashboard
Add a course (admin)	Go to /courses/add, fill form, submit	Course appears in course list
Enroll in a course (student)	Browse courses, click Enroll	Course appears in enrolled list
Drop a course (student)	Click Drop on an enrolled course	Course removed from enrolled list
View enrollment history (student)	Go to `/enrollment/history`	List of all past enrollments
Search courses	Go to `/enrollment/search?q=math`	Filtered course results shown
